

COMP9417 - Machine Learning

Homework 2: Bias, Variance and an application of Gradient Descent

Introduction In this homework we revisit the notion of bias and variance as metrics for characterizing the behaviour of an estimator. We then take a look at a new gradient descent based algorithm for combining different machine learning models into a single, more complex, model.

Points Allocation There are a total of 28 marks.

- Question 1 a): 1 mark
- Question 1 b): 3 marks
- Question 1 c): 2 marks
- Question 1 d): 1 mark
- Question 1 e): 1 mark
- Question 1 f): 1 mark
- Question 2 a): 3 marks
- Question 2 b): 2 marks
- Question 2 c): 6 marks
- Question 2 d): 2 marks
- Question 2 e): 3 marks
- Question 2 f): 2 marks
- Question 2 g): 1 mark

What to Submit

- A **single PDF** file which contains solutions to each question. For each question, provide your solution in the form of text and requested plots. For some questions you will be requested to provide screen shots of code used to generate your answer — only include these when they are explicitly asked for.

- **.py file(s) containing all code you used for the project, which should be provided in a separate .zip file.** This code must match the code provided in the report.
- You may be deducted points for not following these instructions.
- You may be deducted points for poorly presented/formatted work. Please be neat and make your solutions clear. Start each question on a new page if necessary.
- You **cannot** submit a Jupyter notebook; this will receive a mark of zero. This does not stop you from developing your code in a notebook and then copying it into a .py file though, or using a tool such as **nbconvert** or similar.
- We will set up a Moodle forum for questions about this homework. Please read the existing questions before posting new questions. Please do some basic research online before posting questions. Please only post clarification questions. Any questions deemed to be *fishing* for answers will be ignored and/or deleted.
- Please check Moodle announcements for updates to this spec. It is your responsibility to check for announcements about the spec.
- Please complete your homework on your own, do not discuss your solution with other people in the course. General discussion of the problems is fine, but you must write out your own solution and acknowledge if you discussed any of the problems in your submission (including their name(s) and zID).
- As usual, we monitor all online forums such as Chegg, StackExchange, etc. Posting homework questions on these site is equivalent to plagiarism and will result in a case of academic misconduct.
- You may **not** use SymPy or any other symbolic programming toolkits to answer the derivation questions. This will result in an automatic grade of zero for the relevant question. You must do the derivations manually.

When and Where to Submit

- **Due date: Sunday July 5th, 2026 by midnight.** Please note that the forum will not be actively monitored on weekends.
- Late submissions will incur a penalty of 5% per day **from the maximum achievable grade**. For example, if you achieve a grade of 80/100 but you submitted 3 days late, then your final grade will be $80 - 3 \times 5 = 65$. Submissions that are more than 5 days late will receive a mark of zero.
- Submission must be made on **Moodle**, **no exceptions**.

Question 1. Bias of Estimators

Let $\gamma > 0$ and suppose that $X_1, \dots, X_n \stackrel{\text{i.i.d.}}{\sim} N(\gamma, \gamma^2)$. We define:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i,$$
$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2.$$

You may use the following two facts without proof:

(F1) $\bar{X}$ and S^2 are independent.

(F2) $\bar{X}$ and $c_* S$ are both unbiased estimators of γ .¹

What to submit: for all parts (a)-(f), include your working out, either typed or handwritten. For all parts, you must show all working for full credit. Answers without working will receive a grade of zero.

(a) Consider the estimator:

$$T_1 = a\bar{X} + (1-a)c_*S.$$

Show that for any choice of constant a , T_1 is unbiased for γ .

(b) What choice of a gives you the best (in the sense of MSE) possible estimator? Derive an explicit expression for this optimal choice of a . We refer to this estimator as T_1^* .

(c) Consider now a different estimator:

$$T_2 = a_1\bar{X} + a_2(c_*S),$$

and we do not make any assumptions about the relationship between a_1 and a_2 . Find the constants a_1, a_2 explicitly that make T_2 best (from the MSE perspective), i.e. choose a_1, a_2 to minimize $\text{MSE}(T_2) = \mathbb{E}(T_2 - \gamma)^2$. We refer to this estimator as T_2^* .

(d) Show that T_2^* has MSE that is less than or equal to the MSE of T_1^* .

(e) Consider the estimator $V_+ = \max\{0, T_2^*\}$. Show that the MSE of V_+ is smaller than or equal to the MSE of T_2^* .

(f) Let $T_3 = a_1\bar{X} + a_2c_*S + a_3$ be another estimator of γ . Determine the values of a_1, a_2, a_3 that minimize the MSE of T_3 , subject to the constraint that T_3 is unbiased.

Question 2. Gradient Descent for Learning Combinations of Models

In this question, we discuss and implement a gradient descent based algorithm for learning combinations of models, which are generally termed 'ensemble models'. The gradient descent idea is a very powerful one that has been used in a large number of creative ways in machine learning beyond direct minimization of loss functions.

The Gradient-Combination (GC) algorithm can be described as follows: Let $\mathcal{F}$ be a set of base learning algorithms². The idea is to combine the base learners in $\mathcal{F}$ in an optimal way to end up with a good learning algorithm. Let $\ell(y, \hat{y})$ be a loss function, where y is the target, and $\hat{y}$ is the predicted value.³ Suppose we have data (x_i, y_i) for $i = 1, \dots, n$, which we collect into a single data set D_0 . We then set the number of desired base learners to T and proceed as follows:

¹You do not need to worry about knowing or calculating c_* for this question, it is just some constant.

²For example, you could take $\mathcal{F}$ to be the set of all regression models with a single feature, or alternatively the set of all regression models with 4 features, or the set of neural networks with 2 layers etc.

³Note that this set-up is general enough to include both regression and classification algorithms.

(I) Initialize $f_0(x) = 0$ (i.e. f_0 is the zero function.)

(II) For $t = 1, 2, \dots, T$:

(GC1) Compute:

$$r_{t,i} = -\frac{\partial}{\partial f(x_i)} \sum_{j=1}^n \ell(y_j, f(x_j)) \Big|_{f(x_j)=f_{t-1}(x_j), j=1,\dots,n}$$

for $i = 1, \dots, n$. We refer to $r_{t,i}$ as the i -th pseudo-residual at iteration t .

(GC2) Construct a new *pseudo* data set, D_t , consisting of pairs: $(x_i, r_{t,i})$ for $i = 1, \dots, n$.

(GC3) Fit a model to D_t using our base class $\mathcal{F}$. That is, we solve

$$h_t = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n \ell(r_{t,i}, f(x_i))$$

(GC4) Choose a step-size. This can be done by **either** of the following methods:

(SS1) Pick a fixed step-size $\alpha_t = \alpha$

(SS2) Pick a step-size adaptively according to

$$\alpha_t = \arg \min_{\alpha} \sum_{i=1}^n \ell(y_i, f_{t-1}(x_i) + \alpha h_t(x_i)).$$

(GC5) Take the step

$$f_t(x) = f_{t-1}(x) + \alpha_t h_t(x).$$

(III) return f_T .

We can view this algorithm as performing (functional) gradient descent on the base class $\mathcal{F}$. Note that in (GC1), the notation means that after taking the derivative with respect to $f(x_i)$, set all occurrences of $f(x_j)$ in the resulting expression with the prediction of the current model $f_{t-1}(x_j)$, for all j . For example:

$$\frac{\partial}{\partial x} \log(x+1) \Big|_{x=23} = \frac{1}{x+1} \Big|_{x=23} = \frac{1}{24}.$$

- (a) Consider the regression setting where we allow the y -values in our data set to be real numbers. Suppose that we use squared error loss $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$. For round t of the algorithm, show that $r_{t,i} = y_i - f_{t-1}(x_i)$. Then, write down an expression for the optimization problem in step (GC3) that is specific to this setting (you don't need to actually solve it).

What to submit: your working out, either typed or handwritten.

- (b) Using the same setting as in the previous part, derive the step-size expression according to the adaptive approach (SS2).

What to submit: your working out, either typed or handwritten.

- (c) We will now implement the gradient-combination algorithm on a toy dataset from scratch, and we will use the class of decision stumps (depth 1 decision trees) as our base class ($\mathcal{F}$), and squared error loss as in the previous parts.⁴ The following code generates the data and demonstrates plotting the predictions of a fitted decision tree (more details in q2.py):

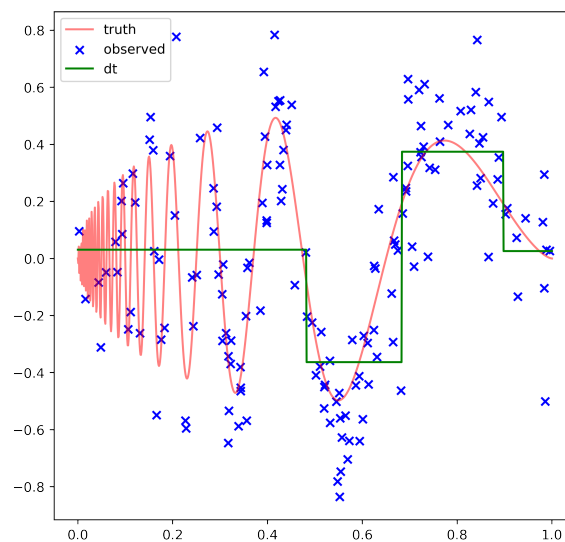
⁴In your implementation, you may make use of `sklearn.tree.DecisionTreeRegressor`, but all other code must be your own. You may use `NumPy` and `matplotlib`, but do not use an existing implementation of the algorithm if you happen to find one.

```

1  np.random.seed(123)
2  X, y = f_sampler(f, 160, sigma=0.2)
3  X = X.reshape(-1,1)
4
5  fig = plt.figure(figsize=(7,7))
6  dt = DecisionTreeRegressor(max_depth=2).fit(X,y) # example model
7  xx = np.linspace(0,1,1000)
8  plt.plot(xx, f(xx), alpha=0.5, color='red', label='truth')
9  plt.scatter(X,y, marker='x', color='blue', label='observed')
10 plt.plot(xx, dt.predict(xx.reshape(-1,1)), color='green', label='dt') # plotting
    example model
11 plt.legend()
12 plt.show()
13

```

The figure generated is



Your task is to generate a 5 x 2 figure of subplots showing the predictions of your fitted gradient-combination model. There are 10 subplots in total, the first should show the model with 5 base learners, the second subplot should show it with 10 base learners, etc. The last subplot should be the gradient-combination model with 50 base learners. Each subplot should include the scatter of data, as well as a plot of the true model (basically, the same as the plot provided above but with your fitted model in place of `dt`). Comment on your results, what happens as the number of base learners is increased? You should do this two times (two 5x2 plots), once with the adaptive step size, and the other with the step-size taken to be $\alpha = 0.1$ fixed throughout. There is no need to split into train and test data here. Comment on the differences between your fixed and adaptive step-size implementations. How does your model perform on the different x-ranges of the data?

What to submit: two 5 x 2 plots, one for adaptive and one for fixed step size, some commentary, and a screen shot of your code and a copy of your code in your .py file.

- (d) Repeat the analysis in the previous question but with depth 2 decision trees as base learners instead. Provide the same plots. What do you notice for the adaptive case? What about the non-adaptive case? *What to submit: two 5 x 2 plots, one for adaptive and one for fixed step size, some commentary, and a copy of your code in your .py file.*
- (e) Now, consider the classification setting where y is taken to be an element of $\{-1, 1\}$. We consider the following classification loss: $\ell(y, \hat{y}) = \log(1 + e^{-y\hat{y}})$. For round t of the algorithm, what is the expression for $r_{t,i}$? Write down an expression for the optimization problem in step (GC3) that is specific to this setting (you don't need to actually solve it).
What to submit: your working out, either typed or handwritten.
- (f) Using the same setting as in the previous part, write down an expression for α_t using the adaptive approach in (SS2). Can you solve for α_t in closed form? Explain.
What to submit: your working out, either typed or handwritten, and some commentary.
- (g) In practice, if you cannot solve for α_t exactly, explain how you might implement the algorithm. Assume that using a constant step-size is not a valid alternative. Be as specific as possible in your answer. What, if any, are the additional computational costs of your approach relative to using a constant step size?
What to submit: some commentary.